Agentic AI

Contents

Foundations & The Autonomous Agent Loop

Single-Agent Frameworks & Tool Calling (ADK, LangChain, SDK)

Multi-Agent Orchestration (Swarm, AutoGen, CrewAI)

Stateful Graph Workflows & Human-in-the-Loop (LangGraph)

Guardrails, Observability, Evaluation & Production Deployment

Foundations & The Autonomous Agent Loop

- **Generative AI vs. Agentic AI:** Moving from prompt-response text generation to multi-step perception-action loops
- **Core Anatomy of an AI Agent:** Brain (LLM), Sensors/Inputs, Working Memory, Tools, and Actuators
- **The ReAct Pattern Deep Dive:** Interleaving Thought → Action → Observation → Reflection
- **Function Calling & Tool Execution Mechanics:** How LLMs output JSON schemas and execute local Python functions using Agent
- **Structured Outputs & Schema Enforcement:** Eliminating hallucinations using Pydantic and JSON validation
- **Local LLM Inference with Ollama:** Running LLM locally, setting LLM parameters
- **Capstone Projects:**
  - Portfolio Risk & Asset Intelligence (ReAct)

Single-Agent Frameworks, Tools & Knowledge Systems



- **Framework Ecosystem:** Comparing Google GenAI SDK (ADK), LangChain, and OpenAI Agent SDK
- **LangChain LCEL:** Declarative pipelines (prompt | model | parser) and tool binding
- **Multi-Tool Orchestration:** Binding multiple tools (Forex APIs, Database queries, Calculator) dynamically
- **Agentic Memory Architectures:** Short-term sliding context buffers vs. Long-term semantic episodic memory
- **Agentic RAG vs. Traditional RAG:** Dynamic query rewriting, self-correction, and hybrid keyword-vector search
- **Grounding & Evidence Citations:** Extracting verified document IDs (HR-POL-042) for compliance
- **Plan-and-Execute Pattern:** Decoupling sequential task decomposition from deterministic tool execution
- Capstone Projects:
  - Expense Auditor (Tool calling)
  - Conversational Buffer (Memory)
  - Enterprise Policy Synthesizer (Agentic RAG)

Multi-Agent Systems, Swarms & Collaborative Debates

- **Why Multi-Agent Systems?:** Specialization, blast-radius isolation, and domain expert personas.
- **Decentralized Swarms & Dynamic Handoffs:** Peer-to-peer delegation without a central coordinator (OpenAI Swarm pattern).
- **Microsoft AutoGen Architecture:** ConversableAgent, GroupChat, and GroupChatManager.
- **Multi-Agent Debate & Self-Consistency:** Having Product, Architecture, Dev, and QA challenge each other to reach consensus.
- **Sandboxed Code Execution:** Safe local Python execution and automated unit testing in agent loops.
- **Hierarchical Crew Orchestration:** Roles, Goals, Backstories, Tasks, and Manager LLM delegation.
- **Actor-Critic & Reflection (Reflexion):** Generating artifacts, running AST linting/critiques, and synthesizing self-correcting patches.
- Capstone Projects:
  - PR Reviewer (Reflection)
  - AutoGen Software Team (Self consistency)
  - Support Swarm (Multi-Agent Collaboration)
  - CrewAI Inventment Crew (Hierarchical Agents)

Stateful Graph Workflows, Human-in-the-Loop & Events

- **State Machine Foundations:** Why acyclic DAGs fail and cyclical state graphs (StateGraph) succeed
- **LangGraph Core:** State schemas, Nodes (actions), Edges (transitions), and Conditional Routers
- **The Hierarchical Supervisor Pattern:** Fan-out parallel subgraphs and state aggregation (operator.add)
- **State Persistence & Checkpoint Recovery:** Saving intermediate snapshots to JSON for failure recovery
- **Human-in-the-Loop (HITL) Architecture:** Pausing execution at high-risk decision gates and resuming
- **Event-Driven AI Pipelines:** Reactive workflows triggered by HTTP webhooks and message queues
- **The Evaluator–Optimizer Pattern:** Two-component generator-discriminator refinement loops for clinical/medical matching
- Capstone Projects:
  - Due Diligence Supervisor (Supervisor)
  - Stateful Loan Underwriting (HIL)
  - Clinical Trial Matcher (Evaluator Optimizer)
  - B2B Lead Enrichment Flow (Event Driven Agents)

Guardrails, Evaluation, Observability & Deployment

- **Production Agent Challenges:** Hallucinations, runaway loops, rate limits, and unauthorized tool calls
- **Deterministic Guardrails & Blast-Radius Containment:** Input/output filtering, pre-flight safety gates, and budget caps
- **Model Context Protocol (MCP):** Universal standardized interface connecting agents to external databases, files, and tools
- **Agent Observability & Tracing:** Step-by-step telemetry, token tracking, and logging with LangSmith, Helicone, and Arize
- **Evaluation & LLM-as-a-Judge:** Evaluating agent trajectory success, tool call accuracy, and RAG faithfulness (RAGAS)
- **Production API Architecture:** Wrapping agents in high-throughput **FastAPI** microservices and Docker containers
- **Interactive UI Dashboards:** Building interactive user interfaces using Streamlit and Rich terminal dashboards
- **Capstone Projects:**
  - Cyber Incident Responder (Plan and Execute)
  - DevOps SRE Self-Healing Engine (Checkpoint and Recovery)

Agentic Frameworks

- OpenAI Agent SDK & Swarms
- Google ADK & GenAI SDK
- Microsoft AutoGen
- LangChain LCEL & Modern Tools
- LangGraph Stateful Graphs & Human-in-the-Loop
- CrewAI Crews and Flows
- n8n workflows

About Instructor

- With **18+ years of crafting technology solutions**, I bring a blend of innovation, leadership, and hands-on expertise
- As the **Associate Technical Director at Sunbeam** and a passionate **Freelance Developer**, I've explored diverse domains, solving complex problems with modern tools and technologies.
- I've had the privilege of developing **many mobile applications** that power experiences on **iOS** and **Android** platforms and crafting dynamic websites with **PHP**, **MEAN**, **and MERN stacks**
- My journey has been fuelled by a deep love for programming languages like **C, C++, Python, JavaScript, TypeScript, Go, Swift, Kotlin and PHP**
- My DevOps journey includes building **CI/CD pipelines using Jenkins, GitHub Actions & ArgoCD**, automating infrastructure with tools like **Terraform & Ansible**, leveraging cloud platforms like **AWS** to create robust, scalable systems and containerizing applications using **Docker** and orchestrating them using **Kubernetes and Docker Swarm**













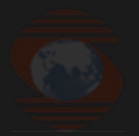
Rules and regulations



- The class will begin at 9:00 pm every day (Please join at least 5 minutes before)
- The classes will be conducted from Monday to Friday
- All the classes will be recorded and will be available in the student's portal next day around 10am
- When you are having the doubts, write them down somewhere
- To avoid the class disturbance, at the end of the theoretical discussion, we will spend time taking the questions all at once
- All the source code will be shared in a GitHub repository

Generative AI

Generative AI



- Generative AI refers to AI systems that can create new content from scratch based on patterns learned from training data
- Rather than just analyzing or classifying existing information, these systems generate entirely new outputs
- **Key capabilities**
  - Creating original content (essays, code, designs) → text | video | audio | image
  - Answering questions in conversational style
  - Summarizing or simplifying complex information
  - Translation between languages
  - Brainstorming and ideation
  - Problem-solving through reasoning

Key characteristics

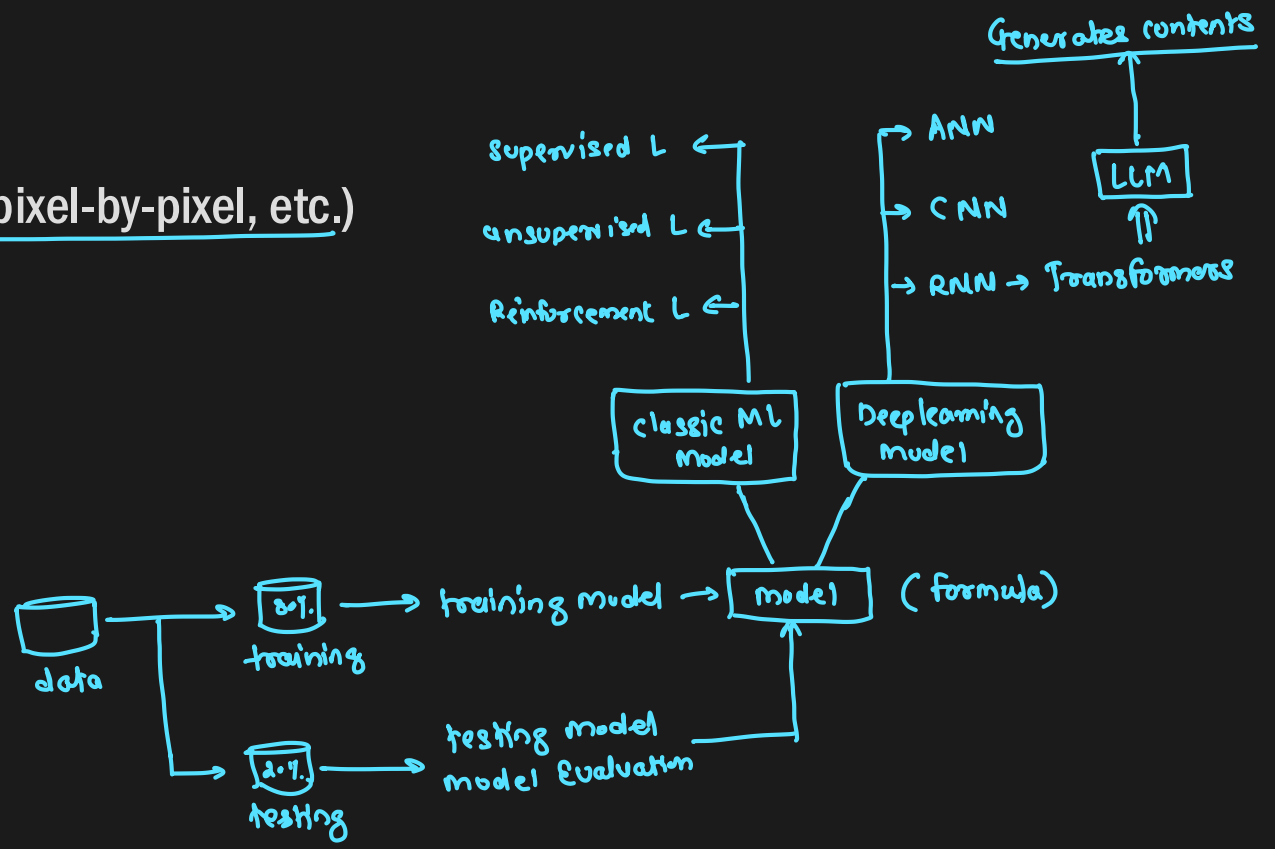
- **Creates new content**
  - The system produces novel outputs—text, images, code, video, audio, etc.—that didn't exist before, rather than retrieving or classifying existing data
- **Learns patterns**
  - It's trained on large datasets to understand patterns, structures, and relationships in that domain
- **Probabilistic**
  - It predicts what should come next (next word, next pixel, next note) based on what came before
- **Flexible**
  - A single model can often generate different outputs for the same input, with some variation

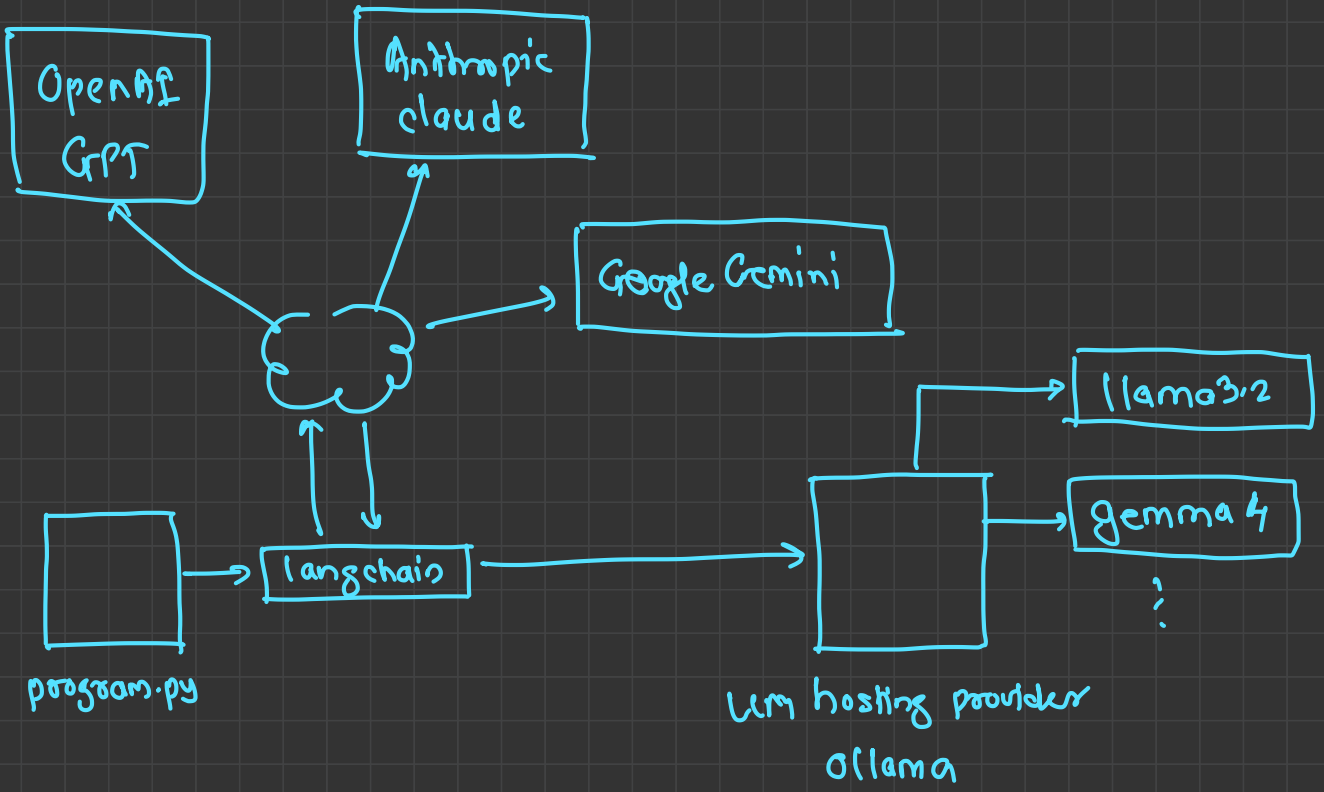
Common types

- **Large Language Models (LLMs)**
  - Generates text
  - Used for writing, answering questions, summarizing, translation, coding
  - Examples: Claude, ChatGPT, Gemini
- **Image generation**
  - Creates images from text descriptions
  - Used for art, design, illustration
  - Examples: DALL-E, Midjourney, Stable Diffusion
- **Code generation**
  - Writes functional code
  - Used for programming assistance
  - Examples: GitHub Copilot, Claude Code
- **Audio/Music generation**
  - Creates speech or music
  - Examples: ElevenLabs, MusicLM

How it works

- **Training**
  - Model learns patterns from massive datasets
- **Encoding**
  - Converts input into mathematical representations
- **Generation**
  - Predicts and generates output token-by-token (or pixel-by-pixel, etc.)
- **Output**
  - Produces novel, contextually relevant content




OpenAI
GPT
Anthropic
claude
Google Gemini
llama3.2
gemma 4
langchain
program.py
LLM hosting provider
ollama

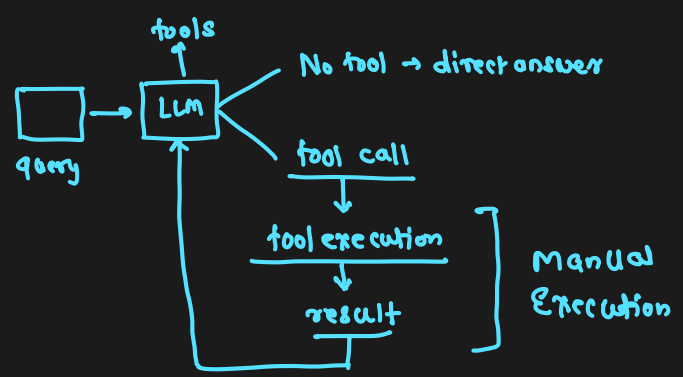
Agentic AI

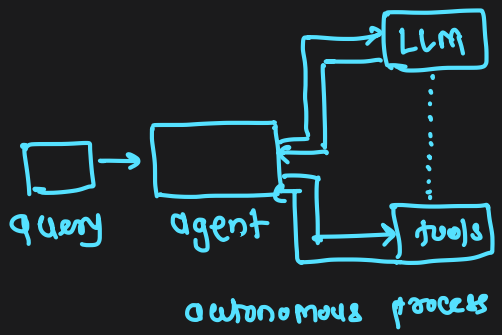
agents

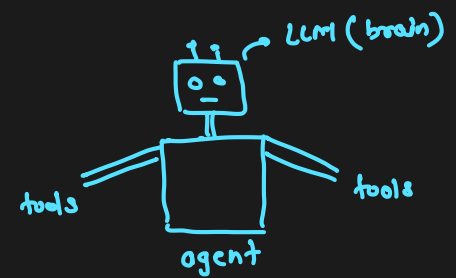
Introduction

automatically

- Agentic AI refers to AI systems that operate autonomously to accomplish specific goals with minimal human intervention
- Rather than just responding to prompts, agentic systems take independent actions over time to solve problems
- Think of the AI assistance as your digital employee
- **Examples**
  - An AI assistant that can autonomously research a topic, organize findings, create a presentation, and email it to you
  - A coding agent that reads your requirements, writes tests, implements features, debugs failures, and pushes code
  - An AI that manages your calendar by reading emails, identifying scheduling needs, and booking meetings
  - A research agent that searches papers, extracts information, synthesizes findings, and flags contradictions







Key Characteristics

- **Goal-oriented**
  - The AI is given an objective and works to achieve it without step-by-step human direction
- **Autonomous decision-making**
  - The system decides what actions to take next, which tools to use, and how to sequence them—rather than waiting for new instructions after each response
- **Tool use**
  - Agentic systems interact with external tools, APIs, and systems to gather information or take actions (search, code execution, database queries, API calls, etc.)
- **Planning & reasoning**
  - They break down complex tasks into subtasks, maintain context across multiple steps, and adapt their approach based on results
- **Iterative refinement**
  - They can evaluate outcomes, adjust strategy, and retry when something doesn't work as intended

Generative vs Agentic AI

- **Generative AI**
  - Focuses on: Output quality and creativity
  - Key question: "Can it produce good content?"
  - Limitation: Produces output and stops—doesn't check if goal was achieved or adapt
  - Example: You ask ChatGPT to "write a blog post about AI," it writes one and waits for your next command
  - Use when: You want creativity, ideas, or content; you want to guide the process

- **Agentic AI**
  - Focuses on: Achieving real-world goals through action
  - Key question: "Can it accomplish the objective autonomously?"
  - Strength: Loops continuously—takes action, evaluates result, adapts, retries until goal is met
  - Example: You tell an agent "write a blog post about AI, optimize it for SEO, create matching social media posts, and schedule them"—it does all that without waiting for instructions between steps
  - Use when: You have a clear goal; you want hands-off execution; the task is complex and multi-step

Agentic AI = Generative AI + Planning + Tool Use + Feedback Loop

Limitations

- **Hallucinations & Accuracy**
  - The AI generates plausible-sounding but false information
- **Cost & Resource Consumption**
  - Agentic systems make many API calls and process lots of tokens
  - **Example**: A simple task might cost $0.02, but an agent looping 5 times to get it right costs $0.10
- **Lack of True Understanding**
  - Agents don't truly understand problems—they pattern-match
  - **Example**: An agent might optimize for the wrong metric because it misunderstood the goal
- **Error Propagation & Brittleness**
  - Errors compound across steps
  - **Example**: If an agent misreads a date format, it schedules everything on the wrong day, and doesn't catch it
- **Difficulty with Complex Reasoning**
  - LLMs are still relatively weak at Multi-step logical reasoning, Complex mathematics and precise calculations etc.
- **Tool Dependency & API Brittleness**
  - Agents depend on external tools
  - **Example**: If the email API is down, your outreach agent stops completely

Challenges

- **Unreliability & Non-Determinism**
  - Same agent, same input, different output. Inconsistent performance makes production deployment risky
- **Cost & Token Inefficiency**
  - Multi-step agentic loops consume far more tokens than necessary, making them economically unviable for many use cases
- **Safety & Uncontrolled Execution**
  - Autonomous systems can take harmful actions at scale without human oversight. Hard to stop or redirect mid-execution
- **Lack of True Reasoning & Understanding**
  - Agents pattern-match rather than actually understand. They struggle with novel problems, complex logic, and real-world constraints
- **Hallucinations & Error Propagation**
  - The AI generates false information or misinterprets real data, and errors cascade through multi-step workflows

Agents

What is Agent ?

- **Traditional AI Agent:** A program that follows a set of rules (if X happens, do Y) to complete a task. It is rigid and reactive
- **Agentic AI Agent:** An AI system that uses a **Large Language Model (LLM)** as its "brain" to reason, plan, and take action autonomously to achieve a complex goal
- **An agent is an autonomous digital entity that perceives its environment, makes independent decisions using generative AI, and takes actions to achieve a specific objective—without needing step-by-step human instructions**
- Examples:
  - **Coding Agent:** Given a GitHub issue, it plans, writes code, debugs errors, tests it, and submits a pull request
  - **Research Agent:** You ask it to write a market report. It autonomously searches the web, reads PDFs, extracts key data, creates a spreadsheet, and writes a summary—all without you telling it which websites to visit
  - **IT Operations Agent:** It monitors server logs, detects a spike in errors, reasons that the database is overloaded, automatically spins up a new cloud server, and reroutes traffic

The Core Components

- Every agentic AI agent is built on a loop of four key functions:
- **Perception**
  - It takes in data from its environment
  - This isn't just text; it can include images, audio, sensor data, database records, or live web searches
- **Brain (The LLM)**
  - This is the engine
  - Instead of just predicting the next word, the LLM acts as the "orchestrator"
  - It engages in **reasoning** (e.g., chain-of-thought) to break a large goal into smaller sub-tasks
- **Planning**
  - The agent dynamically creates a step-by-step plan to achieve the goal. Crucially, it can re-plan mid-course if it encounters an error or new information
- **Action (Tools)**
  - The agent doesn't just talk; it does
  - It connects to external **tools** via APIs
  - This allows it to send emails, update spreadsheets, control robotic arms, query databases, or browse the interne

The Agentic Workflow



- Unlike a standard chatbot that gives one answer and stops, an agentic agent runs in a continuous **"Sense-Think-Act" loop**
- **User gives a goal** (e.g., "Find the cheapest weekend flights to Pune, book a hotel within 2 KMs of the venue, and add it to my calendar")
- **The Agent reasons** and creates a multi-step plan
- **It takes action** (searches flights)
- **It perceives the result** (flight prices)
- **It evaluates** (are these under my budget?)
- **It adjusts its plan** (if not, it searches for alternative dates)
- **It takes the next action** (books the flight, then searches hotels)
- **It stops only when the final goal is achieved**

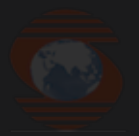
Key Characteristics of an Agentic Agent



- **Autonomy**
  - It operates without human hand-holding. You give it the *what*, and it figures out the *how*.
- **Tool Use**
  - It is "tool-calling." Without tools, an LLM is just a brain in a jar. With tools, it becomes a doer.
- **Reflection & Self-Correction**
  - It can critique its own outputs. For example, it might write code, run it, see that it threw an error, rewrite the code, and run it again until it works
- **Memory**
  - It has **short-term memory** (the current conversation context) and **long-term memory** (a vector database where it stores past interactions, user preferences, or project data to reference later)

Agent Memory

Agent Memory



- **Agent memory** is the mechanism by which an AI agent retains and reuses information beyond a single, immediate input-output exchange
- It's what lets an agent behave with continuity and context-awareness rather than treating every interaction as a blank slate
- An LLM by itself is stateless — each call only "knows" what's in the prompt you send it. Agent memory is the system built around the model that decides:
  - What information to capture from interactions
  - Where to store it
  - When and how to bring it back into context so the agent can use it
- **Why It Matters for Agents Specifically**
  - Unlike a simple chatbot, an agent typically:
    - Executes multi-step tasks (needs to remember what it already tried)
    - Uses tools (needs to remember tool outputs to reason over them later)
    - Operates across sessions (needs to remember user preferences, past decisions, ongoing project state)
    - Sometimes coordinates with other agents (needs shared memory)
  - Without memory, an agent would re-plan from scratch every turn, forget tool results moments after generating them, and have no way to build on past interactions

Types By Persistence / Scope

- **Short-term (working) memory**
  - Lives within a single session or task execution
  - Usually just the conversation/context window itself
  - Includes intermediate reasoning, tool outputs, scratchpad notes
  - Cleared when the session ends
- **Long-term memory**
  - Persists across sessions
  - Requires external storage (vector DB, key-value store, graph DB, relational DB)
  - Needs explicit write/read logic — the agent decides what's worth saving
- **Episodic memory**
  - Stores specific past events/interactions ("what happened")
  - Useful for recalling prior task attempts, user preferences stated once, past decisions and their outcomes
- **Semantic memory**
  - Stores generalized facts/knowledge extracted from experience ("what is true")
  - Often built by summarizing/abstracting episodic memories over time
- **Procedural memory**
  - Stores learned skills, workflows, or successful action sequences
  - E.g., "here's the tool-call sequence that worked for this type of task last time"

Short-term (working) memory — checkpointer

```python
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    "gpt-5",
    checkpointer=InMemorySaver(),   # persists messages per thread_id
)

config = {"configurable": {"thread_id": "session-1"}}
agent.invoke({"messages": [{"role": "user", "content": "Hi, I'm Amit."}]}, config)
```

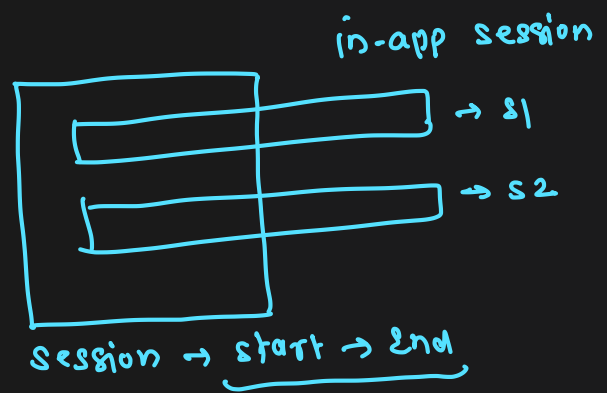
Long-term / persistent memory — store (cross-session)

↳ thread

```python
from langgraph.store.memory import InMemoryStore   # swap for PostgresStore in prod

store = InMemoryStore()

agent = create_agent(
    "gpt-5",
    checkpointer=InMemorySaver(),  → short term
    store=store,   # accessible inside tools via get_store()
)
```



Episodic / semantic memory — namespaced facts in the store



```python
from langgraph.store.base import BaseStore
from langchain.tools import tool

@tool
def remember_preference(user_id: str, fact: str, store: BaseStore):
    """Save a fact learned about the user (episodic/semantic memory)."""
    store.put(("user_memories", user_id), key=fact[:20], value={"fact": fact})
    return "Saved."
```

Procedural memory — reusable tool/skill definitions



```
# Procedural memory = the agent's callable tools/skills themselves,
# often updated by appending a newly-learned successful routine
agent = create_agent(
    "gpt-5",
    tools=[remember_preference, search_db, book_meeting],  # the "how-to" library
    checkpointer=InMemorySaver(),
)
```

Types by Storage Mechanism



- **Buffer memory** — simple sliding window of recent turns, sent raw
- **Summarization memory** — older turns get compressed into running summaries to save context budget
- **Vector store retrieval (RAG-style)** — embeddings of past interactions/facts, retrieved by semantic similarity at query time
- **Key-value / structured stores** — explicit facts stored under keys (like your own memory filesystem here), good for deterministic lookup
- **Knowledge graphs** — entities and relationships, good for multi-hop reasoning over accumulated facts
- **Hybrid** — most production agents combine a few of these (e.g., buffer + vector store + structured profile)

Summarization memory — trims/compresses long context



```python
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
    "gpt-5",
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger={"tokens": 4000},    # summarize once history exceeds this
            keep={"messages": 20},       # retain last 20 raw messages
        )
    ],
)
```

Types by Retrieval Strategy



- Always-inject — certain memory (user profile, system config) is loaded into every prompt regardless of relevance
- Retrieval-on-demand — agent calls a "search memory" tool when it decides it needs past context
- Automatic relevance filtering — a retrieval step runs before the agent responds, injecting only top-k relevant memories
- Reflection-triggered writes — a separate process (sometimes a second LLM call) reviews a finished interaction and decides what's durable enough to write to long-term storage (rather than the agent deciding inline)

Middleware

What is middleware?



- **Middleware is code that plugs into specific points of that loop and runs automatically at each cycle**, without you having to rewrite the agent's core logic
- It's the same concept as HTTP middleware in a web framework (like Express or Django) — a piece of logic that intercepts a request/response pipeline and can inspect, modify, block, or redirect it before it reaches its destination
- Agent middleware does the same thing, but to the model-call / tool-call loop instead of an HTTP request
- Without middleware, if you wanted to, say, trim old messages before every model call, you'd have to fork and rewrite the agent's internal execution graph yourself
- Middleware gives you defined "hook points" to attach that logic instead:
  - **Before the model is called** — e.g., trim/summarize history, inject extra context
  - **After the model responds** — e.g., check the output for policy violations, block a bad answer
  - **Before agent starts / after agent finishes** — e.g., one-time setup, final output validation
  - **Around a tool call** — e.g., catch errors, ask for human approval, redact sensitive data before it reaches a tool

SummarizationMiddleware



- Compresses old messages once a token threshold is hit

```python
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
    "gpt-4o",
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger={"tokens": 4000},
            keep={"messages": 20},
        )
    ],
)
```

HumanInTheLoopMiddleware



- Pauses before risky tool calls, waits for approval

```python
from langchain.agents.middleware import HumanInTheLoopMiddleware

agent = create_agent(
    "claude-sonnet-4-6",
    tools=[read_email, send_email],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={"send_email": True})],
)
```

PIIMiddleware



- Detects/redacts/masks/blocks PII (email, phone, SSN, custom regex)

```python
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    "gpt-4o",
    middleware=[
        PIIMiddleware("email", strategy="redact"),
        PIIMiddleware("credit_card", strategy="mask"),
    ],
)
```

LLMToolEmulator



- Fakes tool outputs via an LLM — for testing without real side effects

```python
from langchain.agents.middleware import LLMToolEmulator

agent = create_agent(
    "gpt-4o",
    tools=[get_weather, send_email],
    middleware=[LLMToolEmulator(tools=["get_weather"])],  # fake this tool's output
)
```

ModelCallLimitMiddleware



- Caps how many model calls an agent can make

```python
from langchain.agents.middleware import ModelCallLimitMiddleware

agent = create_agent(
    "gpt-4o",
    middleware=[ModelCallLimitMiddleware(max_calls=10)],
)
```

ToolCallLimitMiddleware



- Caps how many tool calls an agent can make

```python
from langchain.agents.middleware import ToolCallLimitMiddleware

agent = create_agent(
    "gpt-4o",
    tools=[search_db, book_meeting],
    middleware=[ToolCallLimitMiddleware(max_calls=5)],
)
```

Guardrail



- A **guardrail** is a rule or check that constrains what an AI system is allowed to do or say — it sits between the model's raw capability and the real-world action, catching things that shouldn't happen before they cause harm
- **They typically enforce**
  - Content safety — block toxic, biased, or harmful output
  - Data privacy — prevent leaking PII, secrets, credentials
  - Scope control — keep the agent from answering things it shouldn't (e.g. medical/legal advice)
  - Action safety — require approval before risky operations (deleting data, sending money, sending emails)
  - Format/quality control — reject malformed or hallucinated structured outputs
  - Input validation — filter malicious or manipulative prompts before they reach the model (prompt injection defense)
- **Two Broad Categories**
  - **Input guardrails** — checks applied to what goes into the model (user prompts, retrieved documents, tool results)
  - **Output guardrails** — checks applied to what comes out of the model (its response, or a tool call it wants to make)

Built-in Guardrail Middleware



```python
from langchain.agents.middleware import PIIMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
    "gpt-4o",
    tools=[send_email, delete_record],
    middleware=[
        PIIMiddleware("credit_card", strategy="block"),   # hard-block instead of reda
        HumanInTheLoopMiddleware(interrupt_on={"delete_record": True}),
    ],
)
```

Custom Guardrail — validate output before returning

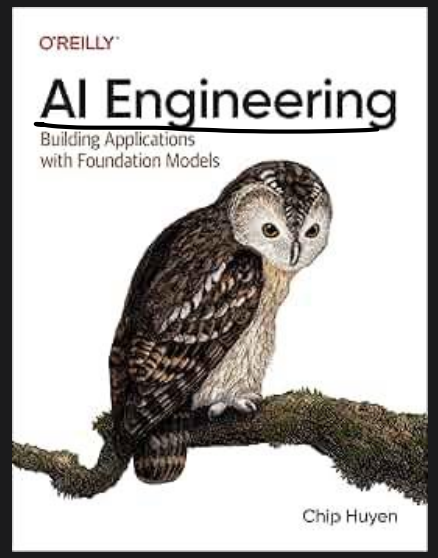


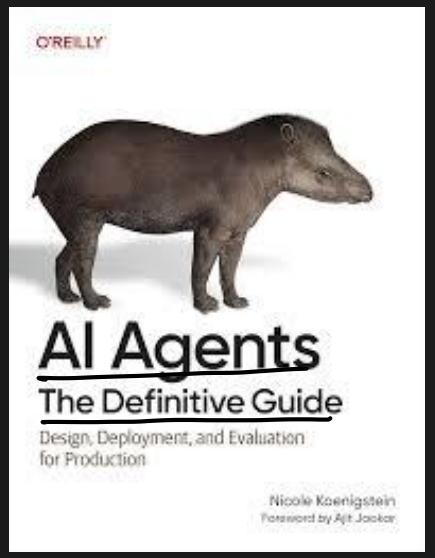
```python
from langchain.agents.middleware import after_agent, AgentState, hook_config
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model

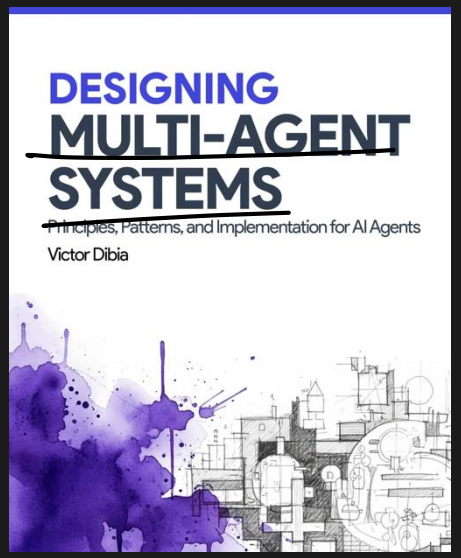
@after_agent
def validate_output(state: AgentState, runtime) -> dict | None:
    last = state["messages"][-1]
    if isinstance(last, AIMessage) and "confidential" in last.content.lower():
        return {"messages": [AIMessage("I can't share that information.")]}
    return None

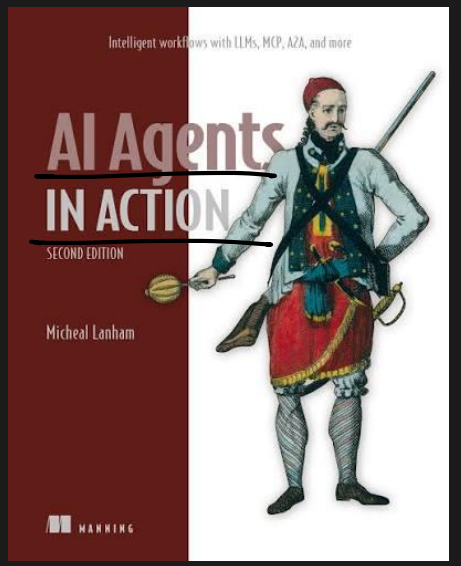
agent = create_agent(
    "gpt-4o",
    tools=[...],
    middleware=[validate_output],
)
```

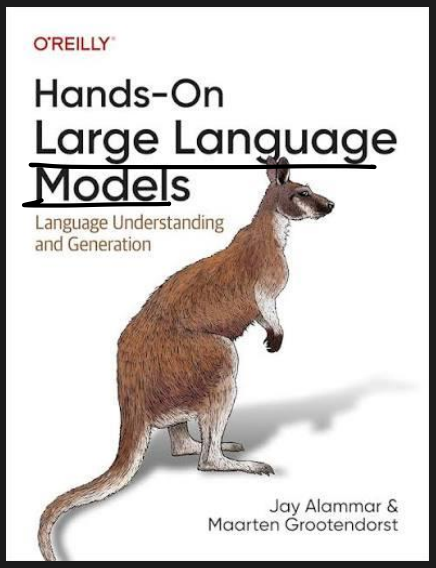
Books


O'REILLY
AI Engineering
Building Applications with Foundation Models
Chip Huyen


O'REILLY
AI Agents
The Definitive Guide
Design, Deployment, and Evaluation for Production
Nicole Koenigstein


DESIGNING
MULTI-AGENT SYSTEMS
Principles, Patterns, and Implementation for AI Agents
Victor Dibia


Intelligent workflows with LLMs, MCP, A2A, and more
AI Agents IN ACTION
SECOND EDITION
Micheal Lanham
MANNING


O'REILLY
Hands-On Large Language Models
Language Understanding and Generation
Jay Alammar &
Maarten Grootendorst

Use Cases

Software Development



- **Code Review & Refactoring Agent**
  - Takes a codebase, identifies issues (bugs, performance, security), refactors code, runs tests, and generates a report
  - Without agency: You'd manually review each file, suggest fixes, wait for feedback
  - With agency: Autonomous improvement cycle
- **Bug Fixing Agent**
  - Reads error logs, reproduces bugs, explores codebase, writes fixes, runs tests, validates solution
  - Saves developers hours of manual debugging

Research & Data Analysis



- **Market Research Agent**
  - Autonomously searches for competitor info, analyzes pricing, collects customer reviews, synthesizes findings into a report
  - Without agency: Researcher manually visits 20+ sites, takes notes, synthesizes
  - With agency: Comprehensive analysis in minutes
- **Academic Paper Summarizer**
  - Takes 50 papers in your field, extracts key findings, identifies gaps, organizes by theme, flags contradictions
  - Saves months of literature review work
- **Financial Analysis Agent**
  - Monitors stock news, extracts market sentiment, correlates with price movements, generates trading signals

Business Operations

- **Sales Lead Qualification & Outreach**
  - Receives leads, researches companies, personalizes outreach emails, tracks responses, and schedules follow-ups
  - Multiplies sales team productivity
- **HR Recruitment Agent**
  - Screens resumes, schedules interviews, sends rejection emails, tracks candidate pipelines
  - Reduces hiring cycle time dramatically
- **Customer Support Agent**
  - Handles inquiries by searching knowledge bases, troubleshooting, escalating complex issues, and closing tickets
  - Works 24/7 without burnout

Education

- **Curriculum Design Agent**
  - Takes learning objectives, researches best practices, designs course structure, creates lesson plans, generates assessments
  - Could dramatically reduce curriculum development time
- **Student Learning Agent**
  - Monitors student progress, identifies knowledge gaps, generates personalized practice problems, provides explanations, tracks improvement
  - Adaptive learning at scale
- **Course Content Generator**
  - Converts textbook chapters into study notes, creates practice questions, generates flashcards, produces video scripts
  - (Similar to your work creating study notes and course materials)

Design Patterns

ReAct (Reason + Act)



- **Concept:** An iterative loop where the agent interleaves reasoning (Thought), action execution (Tool Call), and environment observation (Tool Output) until it arrives at a final answer
- **Flow:** User Request ⟶ Thought (Reasoning) ⟶ Action (Tool Call) ⟶ Observation (Output) ⟶ Final Answer
- **Best used in:** Interactive problem solving, financial ratio calculations, live database queries, and general-purpose tool-assisted reasoning
- **Best Frameworks:** Google GenAI SDK / ADK, LangChain Agents, LlamaIndex
- **Capstone Project:**
  - **Title:** Structured Financial Portfolio Analyst & Risk Assessor
  - **Summary:** Analyzes stock portfolios against real market metrics from JSON files, computes Sharpe Ratios and daily Value-at-Risk (VaR), and generates structured rebalancing plans
  - **Framework Used:** Google GenAI SDK / ADK
  - **Framework Features used:** google-genai function calling, Pydantic structured output validation, and local model inference (llama3.2:3b)

Plan-and-Execute

- **Concept:** Separates planning from execution by having a planner agent generate a comprehensive, sequential execution plan before worker agents execute each phase deterministically
- **Flow:**

  Goal ⟶ [Planner Agent] ⟶ Step 1 ⟶ Step 2 ⟶ Step 3 ⟶ Execution Verification
- **Best used in:** Cybersecurity incident response, multi-step cloud migrations, and structured data extraction
- **Best Frameworks:** Microsoft AutoGen, LangChain Plan-and-Solve
- **Capstone Project:**
  - **Title:** Cybersecurity SOC Incident Response Orchestrator
  - **Summary:** Generates and executes a 4-phase NIST incident response plan: SIEM log detection, threat intel correlation, firewall isolation, and CISO executive briefing
  - **Framework Used:** Microsoft AutoGen
  - **Framework Features used:** Multi-stage sequential execution pipeline, tool calling for firewall and IAM actions, and NIST incident logging

Reflection



- **Concept:** An Actor agent generates an output (e.g., code or text), and an independent Critic agent (or deterministic linter) reviews it against quality/security rules, feeding back specific fixes until the output meets quality standards
- **Flow:**

$$\text{Draft Artifact} \longrightarrow [\text{Critic / AST Scanner}] \xrightarrow{\text{Issues Found}} \text{Actionable Critique} \longrightarrow \text{Refined Final Artifact}$$

- **Best used in:** Automated code generation, AST security auditing, SQL query tuning, and contract drafting.
- **Best Frameworks:** OpenAI Agent SDK, LangGraph, AutoGen.
- **Capstone Project:**
  - **Title:** Autonomous Pull Request Reviewer & Security Bug Fixer
  - **Summary:** Scans PR code using AST analysis to detect SQL injections and unsafe eval calls, provides constructive critiques, and synthesizes a verified git patch.
  - **Framework Used:** OpenAI Agent SDK Pattern
  - **Framework Features used:** AST Python visitor, deterministic security rules engine, and unified diff generator.

Self-Consistency

- **Concept:** A reasoning optimization pattern where the system samples multiple distinct reasoning paths / solutions for the same problem (using varied sampling temperatures, prompts, or diverse agent personas) and selects the most reliable answer via **majority voting, consensus scoring, or automated validation**
- **Flow:**

$$\text{Complex Problem} \longrightarrow \begin{cases} \textit{Reasoning Path 1} \longrightarrow \textit{Candidate Solution A} \\ \textit{Reasoning Path 2} \longrightarrow \textit{Candidate Solution B} \longrightarrow [\textit{Majority Voting / Consensus Engine}] \\ \textit{Reasoning Path 3} \longrightarrow \textit{Candidate Solution A} \end{cases} \longrightarrow \textit{Winning Solution A}$$

- **Best used in:** Complex mathematical calculations, multi-constraint software architecture design, mission-critical code generation, and multi-agent debate consensus
- **Best Frameworks:** Microsoft AutoGen, LangChain / LangGraph, DSPy
- **Capstone Project:**
  - **Title:** Multi-Persona Self-Consistency & Consensus Engine
  - **Summary:** Generates multiple independent technical implementation strategies across distinct personas (Product Manager, Lead Architect, Developer), evaluates test suites in an isolated sandbox, and selects the consensus implementation that passes all criteria.
  - **Framework Used:** Microsoft AutoGen
  - **Framework Features used:** ConversableAgent, multi-turn debate consensus, and local code verification sandbox.

Tool-Calling (Function Execution)



- **Concept:** The agent translates natural language user intents into structured function parameters, invokes local Python functions or external REST APIs, and uses the returned data to complete the task.
- **Flow:** Natural Language Intent ⟶ Schema Argument Extraction ⟶ Function Execution ⟶ Result Grounding
- **Best used in:** Currency conversions, policy rule checks, database lookups, and API integrations.
- **Best Frameworks:** LangChain, OpenAI Agent SDK, Google ADK.
- **Capstone Project:**
  - **Title:** Autonomous Travel & Expense Policy Auditor
  - **Summary:** Converts multi-currency receipts via live Forex APIs and enforces city-by-city per-diem spending caps against policy data in JSON.
  - **Framework Used:** LangChain
  - **Framework Features used:** @tool decorator, LCEL tool binding (bind_tools), and structured argument schemas

Routing & Dynamic Dispatching

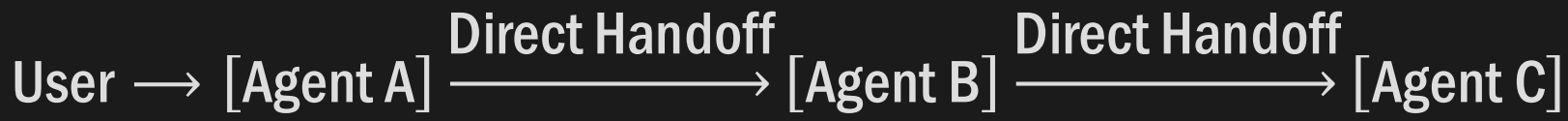
- **Concept:** An intelligent classification gate that evaluates the user prompt and routes execution to the appropriate specialist agent, tool, or downstream workflow to minimize latency and token cost
- **Flow:**

$$\text{Incoming Request} \longrightarrow [\text{Classifier Router}] \begin{cases} \longrightarrow \text{Billing Specialist} \\ \longrightarrow \text{Technical Escalation} \\ \longrightarrow \text{Standard Auto-Reply} \end{cases}$$

- **Best used in:** Multi-tier customer support portals, helpdesk triage, and omnichannel webhook dispatchers
- **Best Frameworks:** OpenAI Agent SDK, n8n AI Workflows, Semantic Router
- **Capstone Project:**
  - **Title:** Customer Support Triage Swarm
  - **Summary:** Ingests support inquiries, routes users to Front-Desk, Billing, or Technical Escalations based on intent, and updates customer accounts
  - **Framework Used:** OpenAI Agent SDK (Swarm Pattern)
  - **Framework Features used:** Swarm Agent definitions, Context Hand-off functions, and typed customer state persistence

Multi-Agent Collaboration (Swarm & Handoffs)

- **Concept:** A decentralized architecture where specialized peer agents pass conversational execution and state directly to one another without bottlenecking through a central orchestrator
- **Flow:**



- **Best used in:** Customer service concierges, e-commerce support, and multi-department internal ticketing
- **Best Frameworks:** OpenAI Agent SDK, Google ADK Swarm
- **Capstone Project:**
  - **Title:** Multi-Tier Customer Care & Handoff Swarm
  - **Summary:** Front-Desk triage smoothly delegates to Billing (for refunds) and SRE (for cluster downtime) with complete conversation history retention
  - **Framework Used:** OpenAI Agent SDK
  - **Framework Features used:** Agent transfer callbacks (transfer_to_billing, transfer_to_tech), shared state objects, and tool registries

Supervisor (Hierarchical Plan-and-Solve)

- **Concept:** A centralized Lead Supervisor agent decomposes a high-level task into sub-tasks, dispatches them across specialized worker agents or subgraphs in parallel, and aggregates their findings into a cohesive report
- **Flow:**

$$\text{Mandate} \longrightarrow [\text{Supervisor}] \begin{cases} \longrightarrow [\text{Macro Worker}] \\ \longrightarrow [\text{Valuation Worker}] \\ \longrightarrow [\text{Moat Worker}] \end{cases} \longrightarrow \text{Synthesized Executive Memo}$$

- **Best used in:** M&A due diligence, institutional equity research, medical tumor boards, and competitive intelligence
- **Best Frameworks:** LangGraph, CrewAI (Hierarchical mode)
- **Capstone Project:**
  - **Title:** Hierarchical Enterprise Due Diligence Supervisor
  - **Summary:** Coordinates three parallel subgraphs analyzing market trends, DCF valuations, and competitor moat data from JSON records into an investment thesis
  - **Framework Used:** LangGraph
  - **Framework Features used:** Supervisor routing nodes, parallel subgraph invocation, and state reduction (operator.add)

Human-in-the-Loop (HITL)

- **Concept:** The agent workflow executes autonomously until it encounters a high-risk decision threshold, where it automatically pauses, persists state, notifies a human operator for review/approval, and resumes upon input
- **Flow:**

$$\text{Auto-Processing} \longrightarrow \text{Threshold Check} \xrightarrow{\text{High Risk}} [\text{PAUSE: Awaiting Human Decision}] \longrightarrow \text{Resume Execution}$$

- **Best used in:** Mortgage loan underwriting, medical prescriptions, wire transfer authorizations, and live database migrations
- **Best Frameworks:** LangGraph, Temporal AI
- **Capstone Project:**
  - **Title:** Stateful Loan Underwriting with Human-in-the-Loop
  - **Summary:** Calculates credit metrics and Debt-to-Income (DTI), triggers a breakpoint interrupt when DTI exceeds safe limits, accepts human underwriter adjustments, and finalizes the loan state
  - **Framework Used:** LangGraph
  - **Framework Features used:** StateGraph, interrupt(), persistent JSON state checkpoints, and conditional resumption edges

Retrieval-Augmented Generation (RAG) & Memory



- **Concept:** Combines conversational memory buffers with semantic/keyword retrieval from knowledge bases to ground agent reasoning in verified enterprise source documents
- **Flow:**
  User Query + Conversation Memory ⟶ Document Retrieval ⟶ Grounded Answer + Official Citations
- **Best used in:** Enterprise HR assistants, compliance policy bots, customer onboarding, and technical documentation lookup
- **Best Frameworks:** LangChain v0.3 LCEL, LlamaIndex
- **Capstone Project:**
  - **Title:** Multi-Source Enterprise Knowledge Synthesizer
  - **Summary:** Answers employee questions regarding remote office stipends, AI security, and travel rules by querying JSON policy documents and attaching verified document citations (HR-POL-042).
  - **Framework Used:** LangChain LCEL
  - **Framework Features used:** ChatPromptTemplate, sliding-window conversation memory buffer, and hybrid document retriever

Evaluator–Optimizer Pattern

- **Concept:** An iterative refinement pattern where one component (the **Optimizer / Generator**) produces a candidate output, while a second independent component (the **Evaluator / Critic**) assesses it against strict criteria, rubrics, or safety constraints. If defects are found, actionable feedback is returned to the Optimizer to refine the output in a closed-loop cycle until the quality threshold is satisfied.
- **Flow:**

Task Request ⟶ [*Optimizer (Generates Candidate)*]

⟶ [*Evaluator (Scores & Screens)*] { *Issues Found (Feedback)* ⟶ [*Optimizer (Refines Artifact)*] ; *Passes All Criteria* ⟶ *Final Validated Output* }

- **Best used in:** High-accuracy code generation with test suites, medical/clinical trial screening against contraindication rules, legal contract drafting, translation refinement, and complex mathematical problem solving.
- **Best Frameworks:** Google GenAI SDK / ADK, LangGraph (Looping Graphs), OpenAI Agent SDK, LlamaIndex Reflexion.
- **Capstone Project:**
  - **Title:** Multimodal Clinical Trial & Pharmacovigilance Matcher
  - **Summary:** The Optimizer agent identifies candidate clinical trials for an oncology patient based on primary diagnosis, while an independent Evaluator screens the candidate list against strict genetic biomarker alterations (EGFR, PD-L1) and drug-drug contraindications (e.g., CYP3A4 inducers) from JSON files to optimize trial eligibility and eliminate patient risk.
  - **Framework Used:** Google GenAI SDK / ADK Pattern
  - **Framework Features used:** Pydantic typed schemas (ClinicalDocument, ClinicalMatchReport), multi-step rule evaluation gates, and local model inference

Panel of Experts (Multi-Agent Debate)



- **Concept:** Multiple agents with distinct personas and conflicting objectives participate in a shared discussion channel to critique, challenge, and collaboratively solve a complex problem
- **Flow:**

  Product Manager ⟷ System Architect ⟷ Senior Developer ⟷ QA Engineer
- **Best used in:** End-to-end software development, architectural reviews, and cross-functional simulations
- **Best Frameworks:** Microsoft AutoGen, CrewAI
- **Capstone Project:**
  - **Title:** Autonomous Agile Software Engineering Team
  - **Summary:** Product Manager writes user stories, Architect creates technical design, Developer writes Python rate limiter code, and QA verifies in an isolated sandbox
  - **Framework Used:** Microsoft AutoGen
  - **Framework Features used:** ConversableAgent, GroupChat, GroupChatManager, and local Python code execution sandbox

Hierarchical Agents (Delegated Role Specialization)

- **Concept:** A structured organization of agents where senior agents delegate domain-specific tasks to specialized junior analyst agents and consolidate their output
- **Flow:**

$$[\textit{Chief Investment Officer}] \begin{cases} \longrightarrow [\textit{Financial Filing Analyst}] \\ \longrightarrow [\textit{DCF Valuation Modeler}] \longrightarrow \textit{Investment Rating} \\ \longrightarrow [\textit{Chief Risk Officer}] \end{cases}$$

- **Best used in:** Hedge fund research, deep market intelligence, complex audit investigations, and enterprise reporting
- **Best Frameworks:** CrewAI, LangGraph
- **Capstone Project:**
  - **Title:** Institutional Market Intelligence & Equity Research Crew
  - **Summary:** Coordinates a Chief Investment Officer, SEC Financial Filing Analyst, and Chief Risk Officer to analyze 10-K data from JSON records and generate a buy/sell rating
  - **Framework Used:** CrewAI
  - **Framework Features used:** Agent, Task, Crew(process=Process.hierarchical), role-based persona prompting, and custom tool bindings

Event-Driven Agents & Reactive Pipelines



- **Concept:** Agents that activate in response to asynchronous triggers (HTTP Webhooks, file events, message queues) to execute chained steps without requiring manual user intervention
- **Flow:**

  Webhook Event ⟶ State Initialization ⟶ Enrichment Step ⟶ Action / Notification
- **Best used in:** Marketing automation, CRM lead qualification, continuous integration pipelines, and real-time fraud alerts.
- **Best Frameworks:** CrewAI Flows, n8n AI Workflows, Temporal.
- **Capstone Project:**
  - **Title:** Event-Driven B2B Prospect Enrichment & Outreach Flow
  - **Summary:** Listens for inbound business leads, scrapes tech signals from Hacker News and JSON records, calculates lead scores, and generates personalized executive email outreach.
  - **Framework Used:** CrewAI Flows
  - **Framework Features used:** @start, @listen decorators, typed state models (Flow[LeadState]), and Pydantic validation.

Checkpoint & State Recovery

- **Concept:** Persisting intermediate execution state at every node transition, allowing long-running multi-step agents to recover from failures or restart exactly where they left off without losing work
- **Flow:**



- **Best used in:** Long-running workflows, multi-day research agents, asynchronous human approvals, and fault-tolerant pipelines
- **Best Frameworks:** LangGraph (SqliteSaver, MemorySaver, JSON State Store), Temporal
- **Capstone Project:**
  - **Title:** Stateful Loan Underwriting Pipeline (code-demos/05_langgraph/app_09_human_in_the_loop_loan_approval)
  - **Summary:** Persists complete underwriting telemetry snapshots into data/loan_applications.json at each stage, ensuring full auditability and zero state loss across interruptions
  - **Framework Used:** LangGraph
  - **Framework Features used:** Durable JSON state checkpointing, thread tracking, and audit trail accumulation